Spring Boot

100 Interview Questions & Answers

Covering Spring Boot 2.x → 3.x → 4.x

✓ Core Concepts & Auto-Configuration
✓ Dependency Injection & Bean Lifecycle
✓ REST APIs, Security & Data Access
✓ Actuator, Testing & Profiles
✓ Spring Boot 3.x — Jakarta EE, Native Images, Virtual Threads
✓ Spring Boot 4.x — API Versioning, Jackson 3, JSpecify, HTTP Service Clients
✓ Microservices, Messaging & Observability

Version Matrix

Spring Boot	Spring Framework	Java Baseline	Jakarta EE	Key Theme
2.7.x	5.x	Java 8+	javax.* (Java EE 8)	Stable LTS, widespread use
3.0 – 3.5	6.x	Java 17+	jakarta.* (EE 9/10)	Modern baseline, GraalVM, Observability
4.0	7.x	Java 17+ (Java 25 preferred)	Jakarta EE 11	Modular, API Versioning, Jackson 3

Note: Spring Boot 4.0 was released November 20, 2025. It requires Jakarta EE 11 (Servlet 6.1), Hibernate 7, Jackson 3, and GraalVM 25 for native images. The Java minimum is still 17, but Java 25 is the recommended version.

Table of Contents

1. Section 1: Spring Boot Fundamentals (Q1–Q15)
2. Section 2: Auto-Configuration & Starters (Q16–Q25)
3. Section 3: Dependency Injection & Bean Lifecycle (Q26–Q35)
4. Section 4: Configuration, Properties & Profiles (Q36–Q45)
5. Section 5: REST API Development (Q46–Q55)
6. Section 6: Spring Data & Database Access (Q56–Q65)
7. Section 7: Spring Security (Q66–Q72)
8. Section 8: Spring Boot Actuator & Observability (Q73–Q80)
9. Section 9: Testing in Spring Boot (Q81–Q87)
10. Section 10: Spring Boot 3.x Features & Changes (Q88–Q93)
11. Section 11: Spring Boot 4.x Features & Changes (Q94–Q100)

Section 1: Spring Boot Fundamentals (Q1–Q15)

Q1: What is Spring Boot and how does it differ from Spring Framework?

Answer: Spring Framework is a comprehensive Java application framework providing IoC, DI, MVC, AOP, and more — but requires significant XML or Java configuration. Spring Boot is an opinionated layer on top of Spring Framework that eliminates most boilerplate configuration through auto-configuration, embedded servers, and starter dependencies. You write less code and get a production-ready application faster.

```
// Spring Boot main class — this single annotation sets everything up
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Q2: What are the four key features of Spring Boot?

Answer: 1) Auto-Configuration: automatically configures Spring beans based on classpath dependencies. 2) Starter Dependencies: curated dependency bundles (e.g. spring-boot-starter-web includes Tomcat, Jackson, Spring MVC). 3) Embedded Server: Tomcat/Jetty/Undertow embedded in the JAR — no external server needed. 4) Production-ready Features: Actuator endpoints for health, metrics, and monitoring out of the box.

Q3: What is @SpringBootApplication and what does it combine?

Answer: @SpringBootApplication is a convenience annotation that combines three annotations: @SpringBootConfiguration (marks as configuration class, alias for @Configuration), @EnableAutoConfiguration (triggers Spring Boot's auto-configuration mechanism), and @ComponentScan (scans for @Component, @Service, @Repository, @Controller in the current package and sub-packages).

Q4: How does Spring Boot auto-configuration work internally?

Answer: When @EnableAutoConfiguration is present, Spring Boot reads all META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports files (in Boot 3+; previously spring.factories) on the classpath. Each file lists auto-configuration classes. These classes use @Conditional annotations (@ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty) to decide whether to create beans. For example, DataSourceAutoConfiguration only activates if a JDBC driver is on the classpath.

```
// Example: DataSource auto-configured only if not manually defined
@ConditionalOnClass(DataSource.class)
@ConditionalOnMissingBean(DataSource.class)
public class DataSourceAutoConfiguration { ... }
```

Q5: What is the difference between @Component, @Service, @Repository, and @Controller?

Answer: All four are specializations of @Component — they all result in the class being registered as a Spring bean. The difference is semantic and functional: @Service marks business logic layer (no extra behavior). @Repository marks data access layer and enables exception translation (converts SQL exceptions to Spring's DataAccessException). @Controller marks MVC controllers for handling HTTP requests. @RestController = @Controller + @ResponseBody.

Q6: What is an embedded server and why is it important?

Answer: Spring Boot embeds a web server (Tomcat by default, or Jetty/Undertow) directly inside the JAR file. This means the application is a self-contained executable — you run it with 'java -jar myapp.jar' without deploying to an external application server. This is essential for microservices and cloud deployments where each service runs independently.

Q7: What is a fat JAR (uber JAR)?

Answer: A fat JAR (also called uber JAR or executable JAR) is a single JAR file that contains the application code AND all its dependencies (including the embedded server). The spring-boot-maven-plugin repackages the JAR to include all dependencies in a nested structure. This makes deployment simple — one file contains everything needed to run the application.

Q8: What is Spring Initializr and what is it used for?

Answer: Spring Initializr (start.spring.io) is a web-based project generator that creates a Spring Boot project skeleton with your chosen dependencies, Java version, build tool (Maven/Gradle), and packaging. It generates a ready-to-import ZIP file with the correct pom.xml/build.gradle, application.properties, and main class. It is also integrated into IDEs like IntelliJ IDEA and Eclipse.

Q9: What is the Spring Boot starter and how does it work?

Answer: A starter is a pre-assembled set of Maven/Gradle dependencies with compatible versions. Instead of manually adding Tomcat, Spring MVC, Jackson, and their exact versions, you add spring-boot-starter-web and Spring Boot handles all transitive dependencies and version compatibility. Common starters: spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-security, spring-boot-starter-test, spring-boot-starter-actuator.

Q10: How do you change the embedded server from Tomcat to Jetty?

Answer: Exclude the Tomcat starter and add the Jetty starter:

```
<!-- In pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

Q11: What is Spring DevTools and what are its main features?

Answer: spring-boot-devtools is a development-time dependency that provides: 1) Automatic restart — detects classpath changes and restarts the application (faster than cold start using two classloaders). 2) LiveReload — triggers browser refresh when resources change. 3) Disabled caching — template caches (Thymeleaf, etc.) are disabled in development. DevTools is automatically disabled in production (not included in fat JARs).

Q12: What is the Banner in Spring Boot and how can you customize it?

Answer: The banner is the ASCII art printed to the console on startup (the Spring logo by default). You can customize it by: 1) Creating src/main/resources/banner.txt with custom ASCII art. 2) Using spring.banner.location property to point to a custom file. 3) Setting spring.main.banner-mode=off to disable it completely. Variables like ${spring.boot.version} can be used inside banner.txt.

Q13: What is CommandLineRunner and ApplicationRunner in Spring Boot?

Answer: Both are functional interfaces used to run code after the Spring Application Context is fully started. CommandLineRunner.run(String... args) receives raw command-line arguments as String array. ApplicationRunner.run(ApplicationArguments args) receives a parsed ApplicationArguments object providing easier access to option args (--key=value) and non-option args.

```
@Component
public class StartupTask implements CommandLineRunner {
```

```
    @Override
    public void run(String... args) throws Exception {
        System.out.println("App started! Args: " + Arrays.toString(args));
    }
}
```

Q14: What is SpringApplication.run() and what does it do?

Answer: SpringApplication.run(MyApp.class, args) is the entry point. It: 1) Creates a SpringApplication instance. 2) Determines the type of ApplicationContext (AnnotationConfigServletWebServerApplicationContext for web apps, AnnotationConfigApplicationContext for non-web). 3) Loads and applies auto-configurations. 4) Starts the embedded server. 5) Fires ApplicationEvents (ApplicationStartedEvent, ApplicationReadyEvent, etc.). 6) Returns the ConfigurableApplicationContext.

Q15: What are ApplicationEvents in Spring Boot and how do you listen to them?

Answer: Spring Boot fires events during startup. Key events: ApplicationStartingEvent (before context), ApplicationEnvironmentPreparedEvent, ApplicationContextInitializedEvent, ApplicationStartedEvent (after context started), ApplicationReadyEvent (application is ready to serve requests), ApplicationFailedEvent. Listen by implementing ApplicationListener<EventType> or using @EventListener annotation.

```
@Component
public class ReadyListener implements ApplicationListener<ApplicationReadyEvent> {
    @Override
    public void onApplicationEvent(ApplicationReadyEvent event) {
        System.out.println("Application is ready!");
    }
}
```

Section 2: Auto-Configuration & Starters (Q16–Q25)

Q16: What are @Conditional annotations? List the key ones.

Answer: @Conditional annotations control whether a bean or configuration class is loaded. Key ones: @ConditionalOnClass (only if class is on classpath), @ConditionalOnMissingClass (only if class is NOT on classpath), @ConditionalOnBean (only if a specific bean exists), @ConditionalOnMissingBean (only if a specific bean does NOT exist — most common in auto-configs), @ConditionalOnProperty (based on a property value), @ConditionalOnExpression (based on SpEL), @ConditionalOnWebApplication, @ConditionalOnNotWebApplication.

Q17: How do you exclude specific auto-configurations?

Answer: Three ways: 1) @SpringBootApplication(exclude = {DataSourceAutoConfiguration.class}) annotation. 2) spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration in application.properties. 3) @EnableAutoConfiguration(exclude = {...}) annotation. This is useful when you don't need a particular auto-config (e.g., you're using an in-memory H2 and don't want a full DataSource configured).

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class,
                                  SecurityAutoConfiguration.class})
public class MyApp { ... }
```

Q18: How do you write your own custom auto-configuration?

Answer: 1) Create a @Configuration class with @Conditional annotations. 2) Register it in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (Spring Boot 3+) or META-INF/spring.factories (Spring Boot 2.x). 3) Use @ConditionalOnMissingBean so your auto-config backs off if the user provides their own bean.

```
@AutoConfiguration
@ConditionalOnClass(MyService.class)
public class MyServiceAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    public MyService myService() {
        return new DefaultMyService();
    }
}
```

Q19: What changed in auto-configuration loading between Spring Boot 2 and Spring Boot 3?

Answer: In Spring Boot 2.x, auto-configuration classes were registered in META-INF/spring.factories under the key org.springframework.boot.autoconfigure.EnableAutoConfiguration. In Spring Boot 3.x, this was changed to a separate file: META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports — one class name per line. The spring.factories file is still supported but the new format is preferred.

Q20: What is the @SpringBootConfiguration annotation?

Answer: @SpringBootConfiguration is a specialization of @Configuration. It is used on the main @SpringBootApplication class. The key addition is that it allows the test framework to find the primary configuration class automatically (TestContext bootstrapping uses @SpringBootConfiguration as the root configuration). It behaves identically to @Configuration otherwise.

Q21: What is spring-boot-starter-parent and what does it provide?

Answer: spring-boot-starter-parent is a special Maven parent POM that provides: 1) Dependency management with pre-configured compatible versions for hundreds of libraries. 2) Plugin management (spring-boot-maven-plugin pre-configured). 3) Java compiler defaults (source/target encoding, Java version). 4) Resource filtering configuration. 5) Default plugin configurations for test,

package, etc. By inheriting it, you avoid specifying versions for most Spring and common dependencies.

Q22: What is the @ConfigurationProperties annotation?

Answer: @ConfigurationProperties binds properties from application.properties/yml to a Java class. Annotate the class with @ConfigurationProperties(prefix="myapp") and Spring will bind all properties starting with 'myapp.' to the class fields. Must be enabled with @EnableConfigurationProperties or @ConfigurationPropertiesScan. Type-safe, supports validation with @Validated, and supports relaxed binding (myapp.server-url or myapp.serverUrl both work).

```
@ConfigurationProperties(prefix = "myapp")
@Validated
public class MyAppProperties {
    @NotBlank
    private String name;
    private int maxConnections = 10;
    // getters and setters
}
```

Q23: What is the difference between @Value and @ConfigurationProperties?

Answer: @Value("${property.key}") injects a single property value into a field. It is simple but scattered — you must add @Value in every class that needs the property. @ConfigurationProperties groups related properties into a single class, supports type-safe binding (nested objects, lists), relaxed binding, and validation. Best practice: use @ConfigurationProperties for complex or shared configuration groups; use @Value for single, simple injections.

Q24: What is Spring Boot's auto-configuration report and how do you see it?

Answer: The auto-configuration report shows which auto-configurations were applied and which were not (and why). To see it: 1) Enable DEBUG logging: logging.level.org.springframework.boot.autoconfigure=DEBUG in application.properties. 2) Use the Actuator /conditions endpoint (previously /autoconfig). 3) Run with --debug JVM argument. The report lists 'Positive matches' (applied), 'Negative matches' (not applied with reason), and 'Exclusions'.

Q25: What is the difference between spring-boot-starter-web and spring-boot-starter-webflux?

Answer: spring-boot-starter-web uses Spring MVC (servlet-based, blocking I/O) with an embedded Tomcat server. It is synchronous/blocking. spring-boot-starter-webflux uses Spring WebFlux (reactive, non-blocking I/O) with an embedded Netty server. WebFlux is built on Project Reactor (Mono/Flux) and supports reactive programming. Key rule: do not mix both starters in one project — if both are present, Spring MVC takes precedence unless explicitly configured otherwise.

Section 3: Dependency Injection & Bean Lifecycle (Q26–Q35)

Q26: What is Inversion of Control (IoC) and Dependency Injection (DI)?

Answer: IoC (Inversion of Control) is a principle where the control of object creation and lifecycle is inverted from the application code to a container (the Spring IoC container). Instead of your code creating its dependencies (new Service()), the container creates and provides them. Dependency Injection is the implementation of IoC — the container injects (provides) dependencies to objects through constructor injection, setter injection, or field injection.

Q27: What are the three types of Dependency Injection in Spring? Which is preferred?

Answer: 1) Constructor Injection: dependencies passed as constructor parameters — PREFERRED. It makes dependencies explicit, enables immutability (final fields), and makes testing easy (no Spring needed). 2) Setter Injection: dependencies set via setter methods — good for optional dependencies. 3) Field Injection: @Autowired directly on a field — convenient but discouraged (hides dependencies, makes testing harder, cannot be used with final). Spring's own code and best practices favor constructor injection.

```
// Preferred: Constructor injection
@Service
public class OrderService {
    private final UserRepository userRepo;
    private final EmailService emailService;

    // @Autowired is optional when there is only one constructor
    public OrderService(UserRepository userRepo, EmailService emailService) {
        this.userRepo = userRepo;
        this.emailService = emailService;
    }
}
```

Q28: What is the Spring Bean scope? List all scopes.

Answer: Bean scope defines the lifecycle and number of instances. Scopes: 1) singleton (default): one instance per ApplicationContext — shared across all requests. 2) prototype: new instance every time the bean is requested. 3) request: one instance per HTTP request (web only). 4) session: one instance per HTTP session (web only). 5) application: one instance per ServletContext (web only). 6) websocket: one instance per WebSocket session. Use @Scope annotation to specify.

```
@Bean
@Scope("prototype")  // or ConfigurableBeanFactory.SCOPE_PROTOTYPE
public MyService myService() {
    return new MyService();
}
```

Q29: What is the difference between @Autowired and @Qualifier?

Answer: @Autowired tells Spring to inject a dependency. By default it injects by type. If multiple beans of the same type exist, Spring throws NoUniqueBeanDefinitionException. @Qualifier resolves this ambiguity by specifying the exact bean name to inject. Use @Primary as an alternative to mark one bean as the default choice when multiple candidates exist.

```
@Autowired
@Qualifier("emailNotificationService")  // specifies which bean
private NotificationService notificationService;

// Alternative: mark one bean as primary
@Bean @Primary
public NotificationService emailNotificationService() { ... }
```

Q30: What is the Bean lifecycle in Spring Boot?

Answer: Bean lifecycle: 1) Instantiation — Spring creates the object. 2) Populate properties — DI happens (@Autowired fields/setters). 3) BeanNameAware, BeanFactoryAware, ApplicationContextAware callbacks. 4) BeanPostProcessor.postProcessBeforeInitialization(). 5) @PostConstruct method called. 6) InitializingBean.afterPropertiesSet(). 7) Custom init-method. 8) BeanPostProcessor.postProcessAfterInitialization(). 9) Bean is ready to use. 10) On shutdown: @PreDestroy, DisposableBean.destroy(), custom destroy-method.

Q31: What are @PostConstruct and @PreDestroy annotations?

Answer: @PostConstruct marks a method to be called after the bean is fully initialized (dependencies injected). Used for setup work like loading data, opening connections, or validating state. @PreDestroy marks a method to be called before the bean is destroyed (application shutdown). Used for cleanup like closing connections or releasing resources. Both are from jakarta.annotation (javax.annotation in Spring Boot 2.x).

```
@Component
public class CacheService {
    @PostConstruct
    public void init() {
        System.out.println("Loading cache from DB...");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Flushing cache to disk...");
    }
}
```

Q32: What is the difference between @Bean and @Component?

Answer: @Component is a class-level annotation for auto-detection via component scanning. Spring creates the bean automatically. @Bean is a method-level annotation inside a @Configuration class. You explicitly create and return the bean object. Use @Bean when: you need to configure third-party library classes (you can't add @Component to them), you need programmatic bean creation, or you have complex initialization logic.

Q33: What is a BeanFactory vs ApplicationContext?

Answer: BeanFactory is the basic Spring IoC container — lazy-loads beans on first request. ApplicationContext extends BeanFactory and adds: eager bean instantiation (by default), MessageSource (i18n), ApplicationEvent publication, AOP integration, and resource loading. In Spring Boot, ApplicationContext (specifically AnnotationConfigServletWebServerApplicationContext for web apps) is always used — BeanFactory alone is rarely used in practice.

Q34: What is @Lazy loading in Spring?

Answer: @Lazy delays the initialization of a bean until it is first requested (instead of at application startup). Applied at class level: @Lazy @Component MyService — the bean is created on first use, not at startup. Can also be used at injection point: @Autowired @Lazy MyService service — creates a proxy, real bean initializes on first method call. Useful for expensive beans not always needed or for breaking circular dependencies.

Q35: How do you handle circular dependencies in Spring Boot?

Answer: A circular dependency occurs when bean A depends on bean B and bean B depends on bean A. Solutions: 1) Refactor to remove the circular dependency (best approach). 2) Use @Lazy on one injection point. 3) Use setter injection instead of constructor injection (allows Spring to create both objects first, then set references). 4) Use @PostConstruct to set the reference. Note: in Spring Boot 2.6+, circular dependencies are detected and fail by default (spring.main.allow-circular-references=true to override — not recommended).

Section 4: Configuration, Properties & Profiles (Q36–Q45)

Q36: What is the order of property precedence in Spring Boot?

Answer: Spring Boot loads properties from many sources in this priority order (highest first): 1) Command-line arguments (--server.port=9090). 2) SPRING_APPLICATION_JSON environment variable. 3) ServletConfig/ServletContext init params. 4) JNDI attributes. 5) Java system properties (System.getProperties()). 6) OS environment variables. 7) application-{profile}.properties. 8) application.properties/yml. 9) @PropertySource annotations. 10) Default properties (SpringApplication.setDefaultProperties). Higher priority sources override lower ones.

Q37: What is the difference between application.properties and application.yml?

Answer: Both serve the same purpose — externalizing configuration — but with different syntax. application.properties uses key=value format with flat keys (spring.datasource.url=). application.yml uses YAML indented format, which is more readable for nested configuration. YAML also supports lists, multi-value properties, and multi-document files (separated by ---). Cannot mix both — if both exist, properties file takes precedence. Many teams prefer YAML for complex configurations.

```
# application.yml example
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: secret
server:
  port: 8080
```

Q38: What are Spring Profiles and how do you use them?

Answer: Profiles allow different configurations for different environments (dev, test, prod). Mark beans with @Profile("prod") to load them only in that profile. Create profile-specific property files: application-dev.properties, application-prod.properties. Activate profiles with: spring.profiles.active=dev in application.properties, or SPRING_PROFILES_ACTIVE=prod environment variable, or command-line --spring.profiles.active=prod.

```
// Bean only loaded in 'prod' profile
@Configuration
@Profile("prod")
public class ProdDataSourceConfig {
    @Bean
    public DataSource dataSource() { /* RDS connection */ }
}

// application-dev.properties
spring.datasource.url=jdbc:h2:mem:testdb

// application-prod.properties
spring.datasource.url=jdbc:mysql://prod-db:3306/mydb
```

Q39: What is the @PropertySource annotation?

Answer: @PropertySource loads a custom properties file into the Spring Environment. Used when you have a properties file outside of application.properties. Added on a @Configuration class: @PropertySource("classpath:custom.properties"). Multiple files: @PropertySources({@PropertySource("file1.properties"), @PropertySource("file2.properties")}). Note: @PropertySource does NOT support YAML files — only .properties files.

Q40: What is @EnableConfigurationProperties?

Answer: @EnableConfigurationProperties(MyProperties.class) enables Spring Boot to process @ConfigurationProperties beans. In Spring Boot 2.2+, you can alternatively use

@ConfigurationPropertiesScan to auto-detect all @ConfigurationProperties classes in the package. You can also annotate the @ConfigurationProperties class with @Component to register it as a bean directly — in that case @EnableConfigurationProperties is not needed.

Q41: How do you read a list or nested object from application.properties?

Answer: For lists: myapp.servers[0]=server1, myapp.servers[1]=server2 binds to List<String> servers. In YAML: servers: - server1 - server2. For nested objects, just use the nested class structure with matching prefixes. @ConfigurationProperties handles all of this automatically with type-safe binding.

```
@ConfigurationProperties(prefix = "myapp")
public class MyAppProperties {
    private List<String> servers = new ArrayList<>();
    private Database database = new Database();

    public static class Database {
        private String host;
        private int port = 5432;
        // getters/setters
    }
}
```

Q42: What is the spring.config.import property?

Answer: Introduced in Spring Boot 2.4, spring.config.import allows importing additional configuration files. You can import local files, classpath resources, or config tree directories. spring.config.import=classpath:extra.properties,optional:file:./custom.yml. The 'optional:' prefix prevents failure if the file doesn't exist. This replaced the older spring.config.additional-location and enables importing config from Kubernetes ConfigMaps mounted as directory trees.

Q43: What is Externalized Configuration in Spring Boot?

Answer: Externalized configuration means keeping configuration outside the application code so the same JAR can run in different environments without recompilation. Spring Boot supports: application.properties/yml (inside JAR), environment variables, command-line arguments, OS environment, config server (Spring Cloud Config), Kubernetes ConfigMaps/Secrets via config tree. This is a key 12-factor app principle.

Q44: What is Relaxed Binding in Spring Boot?

Answer: Relaxed binding means @ConfigurationProperties can bind properties regardless of naming convention. The property myapp.max-connections (kebab-case, recommended), MYAPP_MAX_CONNECTIONS (environment variable format), myapp.maxConnections (camelCase), and myapp.MAX_CONNECTIONS all bind to the same field maxConnections. This makes the configuration more flexible across different deployment environments.

Q45: How do you override properties in tests?

Answer: Multiple approaches: 1) @TestPropertySource(properties = {"my.prop=testValue"}) — inline property override. 2) @TestPropertySource(locations = "classpath:test.properties") — external test properties file. 3) application-test.properties + @ActiveProfiles("test"). 4) @SpringBootTest(properties = {"my.prop=testValue"}) inline. 5) Environment variables set in CI/CD. These allow test environments to use H2 instead of MySQL, mock URLs instead of real ones, etc.

Section 5: REST API Development (Q46–Q55)

Q46: What is the difference between @Controller and @RestController?

Answer: @Controller is used for MVC controllers that return views (HTML pages). The return value is treated as a view name. @RestController = @Controller + @ResponseBody — every method automatically serializes the return value to JSON (or XML). Use @Controller for traditional MVC web apps (Thymeleaf, JSP). Use @RestController for REST APIs that return JSON/XML responses.

Q47: Explain key HTTP mapping annotations in Spring MVC.

Answer: @RequestMapping is the base mapping annotation. The specialized shortcuts: @GetMapping (HTTP GET — fetch data), @PostMapping (HTTP POST — create resource), @PutMapping (HTTP PUT — replace resource), @PatchMapping (HTTP PATCH — partial update), @DeleteMapping (HTTP DELETE — delete resource). All support path, consumes (input media type), and produces (output media type) attributes.

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")                // GET /api/users/1
    public UserDTO getUser(@PathVariable Long id) { ... }

    @PostMapping                            // POST /api/users
    @ResponseStatus(HttpStatus.CREATED)
    public UserDTO createUser(@RequestBody @Valid CreateUserRequest req) { ... }

    @DeleteMapping("/{id}")              // DELETE /api/users/1
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteUser(@PathVariable Long id) { ... }
}
```

Q48: What is @PathVariable vs @RequestParam vs @RequestBody?

Answer: @PathVariable extracts values from the URI path: /users/{id} → @PathVariable Long id. @RequestParam extracts query parameters from the URL: /users?page=1&size=10 → @RequestParam int page. @RequestBody deserializes the HTTP request body (JSON/XML) into a Java object — typically used with POST/PUT. @RequestHeader extracts an HTTP header value.

Q49: How do you handle exceptions globally in Spring Boot?

Answer: Use @ControllerAdvice (or @RestControllerAdvice for REST APIs). Inside it, use @ExceptionHandler methods to handle specific exceptions and return appropriate HTTP responses. This centralizes error handling instead of putting try-catch in every controller.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        // extract field errors from ex.getBindingResult()
        return new ErrorResponse("VALIDATION_ERROR", "Invalid input");
    }
}
```

Q50: What is ResponseEntity and when do you use it?

Answer: ResponseEntity<T> represents the full HTTP response including status code, headers, and body. Use it when you need fine-grained control over the response (custom headers, specific status codes, conditional responses). Alternatives: @ResponseStatus annotation for simple status codes, or return the object directly from @RestController (defaults to 200 OK).

```
@GetMapping("/{id}")
public ResponseEntity<UserDTO> getUser(@PathVariable Long id) {
    return userService.findById(id)
        .map(user -> ResponseEntity.ok(user))
        .orElse(ResponseEntity.notFound().build());
}
```

Q51: How do you validate request input in Spring Boot?

Answer: Use Jakarta Validation (formerly javax.validation) annotations on your request DTOs: @NotNull, @NotBlank, @Size, @Min, @Max, @Email, @Pattern. Add @Valid or @Validated on the @RequestBody parameter to trigger validation. Use @ExceptionHandler(MethodArgumentNotValidException.class) in @RestControllerAdvice to handle validation failures. Configure custom messages in ValidationMessages.properties.

```
public record CreateUserRequest(
    @NotBlank(message = "Name is required")
    String name,
    @Email(message = "Invalid email")
    @NotNull
    String email,
    @Min(value = 18, message = "Must be at least 18")
    int age
) {}

@PostMapping
public UserDTO create(@RequestBody @Valid CreateUserRequest req) { ... }
```

Q52: What is RestTemplate vs WebClient vs RestClient? Which to use?

Answer: RestTemplate (Spring 3+): synchronous HTTP client. Simple to use but deprecated for new code in Spring 5+. WebClient (Spring 5+): reactive, non-blocking HTTP client — returns Mono/Flux. Can also be used in blocking mode. RestClient (Spring Boot 3.2 / Spring Framework 6.1): modern synchronous HTTP client with a fluent API, replacing RestTemplate. Also @HttpExchange (interface-based HTTP clients) is available since Spring 6. Recommendation: RestClient for new blocking code, WebClient for reactive code, @HttpExchange for declarative clients.

Q53: What is CORS and how do you configure it in Spring Boot?

Answer: CORS (Cross-Origin Resource Sharing) is a browser security mechanism preventing requests from one origin (domain/port) to another. In Spring Boot: 1) @CrossOrigin on individual controllers or methods. 2) Global: @Bean WebMvcConfigurer with addCorsMappings(). 3) Security-based: http.cors() in SecurityFilterChain.

```
// Global CORS config
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("https://myapp.com")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowCredentials(true);
    }
}
```

Q54: What is content negotiation in Spring MVC?

Answer: Content negotiation is the mechanism by which the server selects the response format based on what the client accepts. The client sends Accept: application/json (or application/xml). Spring MVC uses ContentNegotiationStrategy to pick an HttpMessageConverter that can produce

that format. Add Jackson for JSON, jackson-dataformat-xml for XML. Use produces attribute on @RequestMapping to restrict which formats an endpoint supports.

Q55: What are HTTP Service Clients (@HttpExchange)? (Spring Boot 3.x/4.x)

Answer: @HttpExchange (Spring Framework 6+, Spring Boot 3+) allows defining HTTP clients as annotated interfaces — similar to Spring Data repositories. Spring creates the implementation automatically. You define the interface with @GetExchange, @PostExchange, etc., then create a bean using RestClient.create() or WebClient. Spring Boot 4.0 adds auto-configuration for these HTTP service clients.

```
@HttpExchange(url = "https://api.example.com")
public interface UserApiClient {
    @GetExchange("/users/{id}")
    UserDTO getUser(@PathVariable Long id);

    @PostExchange("/users")
    UserDTO createUser(@RequestBody CreateUserRequest req);
}

// Spring Boot 4 auto-configures this if configured via properties
// Or manually:
@Bean
UserApiClient userApiClient(RestClient.Builder builder) {
    RestClient restClient = builder.baseUrl("https://api.example.com").build();
    return HttpServiceProxyFactory.builderFor(RestClientAdapter.create(restClient))
           .build().createClient(UserApiClient.class);
}
```

Section 6: Spring Data & Database Access (Q56–Q65)

Q56: What is Spring Data JPA and what does it provide?

Answer: Spring Data JPA is an abstraction layer over JPA (Jakarta Persistence API) and Hibernate ORM. It provides: 1) Repository interfaces (CrudRepository, JpaRepository, PagingAndSortingRepository) that auto-generate CRUD SQL. 2) Derived query methods — method names like findByEmailAndAge() are translated to SQL. 3) @Query annotation for JPQL or native SQL. 4) Pagination and sorting. 5) Auditing (@CreatedDate, @LastModifiedDate). Eliminates 90% of boilerplate DAO code.

Q57: What is the difference between CrudRepository, JpaRepository, and PagingAndSortingRepository?

Answer: CrudRepository: basic CRUD operations (save, findById, findAll, delete, count). PagingAndSortingRepository: extends CrudRepository, adds findAll(Pageable) and findAll(Sort). JpaRepository: extends PagingAndSortingRepository, adds JPA-specific methods: flush(), saveAndFlush(), deleteInBatch(), getById(). JpaRepository is the most commonly used as it includes all operations from both parents.

Q58: How do derived query methods work in Spring Data JPA?

Answer: Spring Data parses the method name and generates JPQL automatically. Keywords: findBy, readBy, queryBy, getBy (select), countBy, existsBy, deleteBy/removeBy. Conditions: And, Or, Equals, Like, NotLike, Between, LessThan, GreaterThan, OrderBy, IsNull, IsNotNull, In, NotIn, StartingWith, EndingWith, Containing.

```
public interface UserRepository extends JpaRepository<User, Long> {
    // SELECT * FROM users WHERE email = ?
    Optional<User> findByEmail(String email);

    // SELECT * FROM users WHERE age BETWEEN ? AND ? ORDER BY name
    List<User> findByAgeBetweenOrderByName(int minAge, int maxAge);

    // SELECT COUNT(*) WHERE active = true
    long countByActiveTrue();

    // Custom JPQL
    @Query("SELECT u FROM User u WHERE u.name LIKE %:name%")
    List<User> searchByName(@Param("name") String name);
}
```

Q59: What is @Transactional and how does it work?

Answer: @Transactional marks a method or class to run within a database transaction. Spring creates a proxy around the bean — when the method is called, the proxy starts a transaction; on successful return it commits; on exception it rolls back (by default only for unchecked exceptions — RuntimeException and Error). Attributes: propagation (REQUIRED, REQUIRES_NEW, etc.), isolation (READ_COMMITTED, etc.), readOnly=true (optimization hint), rollbackFor (also rollback on checked exceptions).

```
@Service
public class TransferService {
    @Transactional(rollbackFor = Exception.class,
                   isolation = Isolation.READ_COMMITTED)
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        accountRepo.debit(fromId, amount);
        accountRepo.credit(toId, amount);
        // If credit throws, the debit is also rolled back
    }
}
```

Q60: What is the N+1 query problem and how do you solve it?

Answer: The N+1 problem occurs when loading N parent entities generates N additional queries to load their child collections — one per parent. For example, loading 100 orders triggers 100 queries to load each order's items. Solutions: 1) @EntityGraph or JOIN FETCH in JPQL to eagerly load in one query. 2) @BatchSize(size=50) to load children in batches. 3) @Query with custom JPQL JOIN FETCH. 4) Use @ManyToOne(fetch=EAGER) carefully. @Transactional on service method also helps by keeping the session open.

Q61: What is the difference between FetchType.LAZY and FetchType.EAGER?

Answer: LAZY: the associated entity/collection is NOT loaded from the database until it is accessed in code. This is the default for @OneToMany and @ManyToMany. Efficient but requires an open Hibernate session when accessed. EAGER: the associated entity is loaded immediately with the parent query (via JOIN or extra query). Default for @ManyToOne and @OneToOne. Can cause performance issues if associations are large. Best practice: use LAZY everywhere, fetch eagerly only when needed via @Query with JOIN FETCH.

Q62: What is Spring Data's Pageable and how do you implement pagination?

Answer: Pageable is a Spring Data interface for pagination and sorting. Pass it to repository methods: Page<User> findAll(Pageable pageable). Controllers accept Pageable as a method parameter (Spring auto-parses ?page=0&size=10&sort=name,asc). The returned Page<T> contains the data, total elements, total pages, current page info.

```
@GetMapping("/users")
public Page<UserDTO> getUsers(Pageable pageable) {
    // GET /users?page=0&size=10&sort=name,asc
    return userRepository.findAll(pageable)
                         .map(userMapper::toDTO);
}
```

Q63: What is Spring Data's @Query annotation?

Answer: @Query lets you define custom JPQL or native SQL queries on repository methods. Use nativeQuery=true for native SQL. Use @Modifying for UPDATE/DELETE queries (also needs @Transactional). Named parameters with @Param or positional ?1, ?2.

```
// JPQL query
@Query("SELECT u FROM User u WHERE u.status = :status AND u.age > :minAge")
List<User> findActiveUsersOlderThan(@Param("status") String status,
                                    @Param("minAge") int minAge);

// Native SQL
@Query(value = "SELECT * FROM users WHERE email LIKE %:domain",
       nativeQuery = true)
List<User> findByEmailDomain(@Param("domain") String domain);

// Bulk update — requires @Modifying
@Modifying
@Query("UPDATE User u SET u.status = :status WHERE u.lastLogin < :date")
int deactivateInactiveUsers(@Param("status") String status,
                            @Param("date") LocalDate date);
```

Q64: What are Flyway and Liquibase? How do you use them with Spring Boot?

Answer: Flyway and Liquibase are database migration tools that manage schema changes using versioned scripts, ensuring all environments have the same schema. Flyway uses SQL migration files named V1__Initial_schema.sql, V2__Add_users_table.sql in src/main/resources/db/migration. Spring Boot auto-configures both when their dependency is on the classpath. Properties: spring.flyway.enabled=true, spring.flyway.locations=classpath:db/migration. Liquibase uses XML/YAML changelog files.

Q65: What is JdbcTemplate and when do you use it instead of JPA?

Answer: JdbcTemplate is Spring's JDBC abstraction that handles connection management, exception translation, and resource cleanup. Use it when: JPA's abstraction is too heavy, you need fine-grained SQL control, performance is critical (no entity overhead), working with stored procedures, or the schema doesn't map cleanly to entities. Spring Boot 3.2+ also added JdbcClient — a fluent, modern alternative to JdbcTemplate with a cleaner API.

```
@Repository
public class UserDao {
    private final JdbcTemplate jdbcTemplate;

    public List<User> findByStatus(String status) {
        return jdbcTemplate.query(
            "SELECT * FROM users WHERE status = ?",
            (rs, row) -> new User(rs.getLong("id"), rs.getString("name")),
            status
        );
    }
}
```

Section 7: Spring Security (Q66–Q72)

Q66: How does Spring Security work in Spring Boot?

Answer: Adding spring-boot-starter-security auto-configures security with defaults: all endpoints require authentication, a login form is provided, a default user 'user' is created with a generated password (printed to console). The security filter chain (SecurityFilterChain) processes every HTTP request through a series of filters. The key filter is UsernamePasswordAuthenticationFilter for form login, or others for JWT/OAuth2. You customize by providing a @Bean SecurityFilterChain.

Q67: What is the SecurityFilterChain and how do you configure it?

Answer: SecurityFilterChain is the modern way (since Spring Security 5.4, Spring Boot 2.7+) to configure HTTP security — replacing the deprecated WebSecurityConfigurerAdapter. You define a @Bean that returns SecurityFilterChain, configuring authorization rules, authentication mechanisms, session management, CSRF, etc.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())  // disable for REST APIs
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

Q68: What is JWT and how do you implement JWT authentication in Spring Boot?

Answer: JWT (JSON Web Token) is a compact, self-contained token for stateless authentication. Structure: Header.Payload.Signature (Base64 encoded, dot-separated). Flow: 1) Client sends credentials → server validates and returns JWT. 2) Client stores JWT (localStorage/cookie). 3) Each subsequent request includes JWT in Authorization: Bearer <token> header. 4) A custom JwtFilter extracts, validates, and sets the SecurityContext. Libraries: java-jwt (Auth0), JJWT (io.jsonwebtoken).

Q69: What is the difference between Authentication and Authorization?

Answer: Authentication is verifying IDENTITY — 'Who are you?' (login with username/password, JWT validation). Authorization is verifying PERMISSIONS — 'What are you allowed to do?' (can this user access /admin/users?). In Spring Security: AuthenticationManager handles authentication, AccessDecisionManager (or AuthorizationManager in Spring Security 6) handles authorization. Annotations: @PreAuthorize, @PostAuthorize, @Secured for method-level authorization.

Q70: What is CSRF protection and when should you disable it?

Answer: CSRF (Cross-Site Request Forgery) is an attack where a malicious site tricks a logged-in user's browser into making an unauthorized request. Spring Security enables CSRF protection by default using a CSRF token in forms. Disable CSRF for REST APIs that use stateless authentication (JWT tokens sent in Authorization header), because REST clients (mobile apps, frontend SPAs using Bearer tokens) are not vulnerable to CSRF. Keep CSRF enabled for traditional web apps with session-based authentication.

Q71: What is OAuth2 and how does Spring Boot support it?

Answer: OAuth2 is an authorization protocol allowing third-party apps to access resources on behalf of a user. Spring Boot provides spring-boot-starter-oauth2-client for acting as an OAuth2 client (login with Google/GitHub), spring-boot-starter-oauth2-resource-server for protecting APIs with OAuth2 JWT tokens, and spring-authorization-server for building your own authorization server. Configure in application.properties with spring.security.oauth2.client.registration.*

Q72: What is @PreAuthorize and @PostAuthorize?

Answer: @PreAuthorize checks authorization before the method executes using Spring Expression Language (SpEL). @PostAuthorize checks after execution (can access returnObject). Enable with @EnableMethodSecurity on a @Configuration class. Common expressions: hasRole('ADMIN'), hasAnyRole('ADMIN','MANAGER'), isAuthenticated(), hasAuthority('READ_PRIVILEGE'), #id == authentication.principal.id (user can only access their own data).

```
@EnableMethodSecurity
@Configuration
public class MethodSecurityConfig { }

@Service
public class UserService {
    @PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
    public UserDTO getUser(Long userId) { ... }

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteUser(Long userId) { ... }
}
```

Section 8: Spring Boot Actuator & Observability (Q73–Q80)

Q73: What is Spring Boot Actuator and what does it provide?

Answer: Spring Boot Actuator provides production-ready monitoring and management endpoints over HTTP and JMX. Key endpoints: /actuator/health (app health), /actuator/info (app metadata), /actuator/metrics (JVM/HTTP metrics), /actuator/env (environment properties), /actuator/beans (all Spring beans), /actuator/mappings (all HTTP mappings), /actuator/loggers (view/change log levels at runtime), /actuator/threaddump, /actuator/heapdump. Add spring-boot-starter-actuator to enable.

Q74: How do you secure Actuator endpoints?

Answer: Actuator endpoints should be secured in production. Options: 1) Expose only specific endpoints: management.endpoints.web.exposure.include=health,info. 2) Require authentication: configure Spring Security to restrict /actuator/** to ADMIN role. 3) Run on a different port: management.server.port=8081 (so it's not accessible externally). 4) Use management.endpoints.web.base-path to change the path. By default only /health and /info are exposed over HTTP; all others are disabled.

```
# application.properties
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=when-authorized
management.server.port=8081
```

Q75: How do you create a custom health indicator?

Answer: Implement the HealthIndicator interface and register it as a @Component. Spring Boot auto-detects it and includes it in /actuator/health.

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean isDbUp = checkDatabaseConnection();
        if (isDbUp) {
            return Health.up().withDetail("db", "Connected").build();
        }
        return Health.down().withDetail("error", "Cannot connect").build();
    }
}
```

Q76: What is Micrometer and how is it used with Spring Boot?

Answer: Micrometer is the metrics library that Spring Boot Actuator uses. It provides a vendor-neutral API for recording application metrics (counters, gauges, timers, distribution summaries). Micrometer integrates with monitoring backends: Prometheus, Grafana, Datadog, New Relic, Influx. Add the micrometer-registry-prometheus dependency to expose metrics in Prometheus format at /actuator/prometheus. Spring Boot auto-instruments HTTP requests, JVM, DataSource, cache, and more.

Q77: What is Distributed Tracing in Spring Boot 3+?

Answer: Distributed tracing tracks a request as it flows through multiple microservices. Spring Boot 3 introduced built-in support via Micrometer Tracing (replacing Spring Cloud Sleuth). Each request gets a trace ID and each operation gets a span ID. Logs automatically include traceId and spanId. Spring Boot auto-configures Brave (Zipkin) or OpenTelemetry exporters. Add micrometer-tracing-bridge-otel and opentelemetry-exporter-zipkin or OTLP exporter. Spring Boot 4 adds an OpenTelemetry Starter with better integration.

Q78: What is @Async and how do you use it in Spring Boot?

Answer: @Async runs a method in a separate thread asynchronously. Enable with @EnableAsync on a configuration class. The method must be in a Spring-managed bean and called from outside the bean (not self-invocation). Returns void for fire-and-forget, or CompletableFuture<T>/Future<T> for async with result. Spring Boot auto-configures a ThreadPoolTaskExecutor as the default executor. Configure thread pool with spring.task.execution.pool.* properties.

```
@Service
public class EmailService {
    @Async
    public CompletableFuture<String> sendEmail(String to) {
        // Runs in separate thread
        emailProvider.send(to);
        return CompletableFuture.completedFuture("Sent to " + to);
    }
}
```

Q79: What is @Scheduled in Spring Boot?

Answer: @Scheduled enables scheduling of methods to run at fixed intervals or using cron expressions. Enable with @EnableScheduling. Annotations: @Scheduled(fixedRate=5000) — runs every 5 seconds. @Scheduled(fixedDelay=5000) — 5 seconds after last execution finishes. @Scheduled(cron="0 0 * * * *") — every hour at minute 0 (standard cron). By default runs in a single thread — configure TaskScheduler for parallel execution.

```
@Component
public class ReportScheduler {
    @Scheduled(cron = "0 0 8 * * MON-FRI")  // 8 AM weekdays
    public void generateDailyReport() { ... }

    @Scheduled(fixedDelay = 60_000)  // every minute
    public void cleanupExpiredSessions() { ... }
}
```

Q80: What is the @Observed annotation and Observability in Spring Boot 3+?

Answer: @Observed (Spring Boot 3.x) adds automatic tracing AND metrics to a method or class. When placed on a method, it creates a span (trace) and records a timer (metric) automatically — without manual Micrometer code. Requires the Micrometer Tracing starter and AspectJ. Spring Boot 4 further improves this with an OpenTelemetry starter and better OTLP integration, making distributed tracing configuration easier.

```
@Service
public class OrderService {
    @Observed(name = "order.processing",
              contextualName = "processOrder")
    public Order processOrder(OrderRequest req) {
        // Automatically traced + metrics recorded
        return orderRepository.save(createOrder(req));
    }
}
```

Section 9: Testing in Spring Boot (Q81–Q87)

Q81: What is @SpringBootTest and what does it do?

Answer: @SpringBootTest loads the full Spring application context for integration testing. By default starts an embedded server (webEnvironment=RANDOM_PORT or DEFINED_PORT). Supports: @Autowired for injecting beans, @MockBean for replacing beans with Mockito mocks, @SpyBean for partial mocks. It is slower than unit tests because it starts the full context — use for integration tests; use @WebMvcTest/@DataJpaTest for slice tests.

Q82: What is @WebMvcTest and when do you use it?

Answer: @WebMvcTest loads only the web layer (controllers, filters, @ControllerAdvice) without the full context. Use it for unit-testing controllers without a real database or service implementation. Combine with @MockBean to mock services. Uses MockMvc for simulating HTTP requests without starting a real server — very fast.

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean UserService userService;

    @Test
    void getUser_returnsUser() throws Exception {
        given(userService.findById(1L)).willReturn(new UserDTO(1L, "Alice"));

        mockMvc.perform(get("/api/users/1"))
               .andExpect(status().isOk())
               .andExpect(jsonPath("$.name").value("Alice"));
    }
}
```

Q83: What is @DataJpaTest?

Answer: @DataJpaTest loads only the JPA layer (entities, repositories, Hibernate) with an in-memory H2 database. It does NOT start the web layer. Transactions are rolled back after each test by default. Use it to test repositories and JPA queries without a full application context. Faster than @SpringBootTest for database-layer tests. Use @AutoConfigureTestDatabase(replace=NONE) to use the real database instead of H2.

Q84: What is MockMvc and how is it different from TestRestTemplate?

Answer: MockMvc tests the web layer without starting a real HTTP server — it simulates requests directly through the DispatcherServlet. Fast, great for controller unit tests. TestRestTemplate starts a real embedded server and makes actual HTTP calls — better for end-to-end integration tests. Use MockMvc with @WebMvcTest (no server) or @SpringBootTest(webEnvironment=MOCK). Use TestRestTemplate with @SpringBootTest(webEnvironment=RANDOM_PORT).

Q85: What is @MockBean vs @Mock in Spring Boot tests?

Answer: @Mock (Mockito): creates a Mockito mock but does NOT put it in the Spring context. Used in pure unit tests without Spring. @MockBean (Spring Boot Test): creates a Mockito mock AND replaces the bean in the Spring Application Context. Use it when you need a mock in an integration test (the Spring context must know about it — e.g., in @SpringBootTest or @WebMvcTest). @SpyBean is similar but wraps a real bean, letting you spy on it while keeping real behavior.

Q86: What are Testcontainers and how do you use them with Spring Boot?

Answer: Testcontainers is a Java library that starts real Docker containers during tests — real PostgreSQL, Redis, Kafka, etc. Spring Boot 3.1+ has native Testcontainers integration with

@ServiceConnection, which auto-configures the datasource URL from the container. No need to manually override spring.datasource.url.

```
@SpringBootTest
@Testcontainers
class UserRepositoryTest {
    @Container
    @ServiceConnection  // Spring Boot 3.1+ auto-configures DataSource from this
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:15");

    @Autowired UserRepository userRepository;

    @Test
    void saveUser_persistsToRealDB() {
        User saved = userRepository.save(new User("Alice", "alice@test.com"));
        assertThat(saved.getId()).isNotNull();
    }
}
```

Q87: What is the difference between unit tests, integration tests, and end-to-end tests in Spring Boot?

Answer: Unit tests: test a single class in isolation with no Spring context (JUnit 5 + Mockito). Fast, deterministic, no DI needed. Integration tests: test multiple layers together with a Spring context (@WebMvcTest, @DataJpaTest, @SpringBootTest). Real beans with some mocks. Slower.
End-to-end tests: test the full application stack with a real server and real database (Testcontainers + TestRestTemplate). Slowest, most realistic. Best practice: many unit tests (fast feedback), fewer integration tests, few E2E tests (pyramid).

Section 10: Spring Boot 3.x Features & Changes (Q88–Q93)

Key changes: Java 17 baseline, jakarta.* namespace, Native Images, Virtual Threads, Observability

Q88: What are the major changes in Spring Boot 3.0 compared to Spring Boot 2.x?

Answer: Spring Boot 3.0 (November 2022) is a major breaking change: 1) Minimum Java 17 required. 2) javax.* packages renamed to jakarta.* (Jakarta EE 9+). 3) Spring Framework 6 as the base. 4) Native image support via GraalVM AOT compilation (first-class support). 5) Spring Security configuration model changed (no more WebSecurityConfigurerAdapter). 6) spring.factories replaced by AutoConfiguration.imports. 7) Built-in Observability (Micrometer Tracing replacing Sleuth). 8) Hibernate 6. 9) Spring Data 3.

Q89: What is the javax to jakarta namespace change and why does it matter?

Answer: Java EE's specifications moved to the Eclipse Foundation under the Jakarta EE project. Oracle retained the javax.* namespace, so Jakarta had to rename all packages to jakarta.*. Key changes: javax.servlet → jakarta.servlet, javax.persistence → jakarta.persistence, javax.validation → jakarta.validation, javax.annotation → jakarta.annotation. Any Spring Boot 3.x project referencing javax.* imports will fail to compile. All existing code using Java EE APIs must update every import.

Q90: What is GraalVM Native Image and how does Spring Boot 3 support it?

Answer: GraalVM Native Image compiles Java applications to native machine code at build time (AOT — Ahead-of-Time compilation). Native images start extremely fast (milliseconds vs seconds) and use much less memory than JVM applications. Spring Boot 3 supports native images via the spring-boot-maven-plugin/gradle-plugin with ./mvnw spring-boot:build-image -Pnative. Challenges: reflection, dynamic class loading, and proxies need hints. Spring's AOT processor automatically generates most hints.

```
# Build a native image
./mvnw -Pnative spring-boot:build-image

# Or native executable
./mvnw -Pnative native:compile

# Run — starts in milliseconds
./target/myapp
```

Q91: How do you enable Virtual Threads in Spring Boot 3.2?

Answer: Spring Boot 3.2 (Spring Framework 6.1) adds first-class Virtual Thread support. One property enables them for the embedded Tomcat server (and Spring's TaskExecutor): spring.threads.virtual.enabled=true. This switches Tomcat's thread model to use virtual threads per request, allowing millions of concurrent connections without the memory overhead of platform threads.

```
# application.properties
spring.threads.virtual.enabled=true

# That's it! Tomcat now uses virtual threads (Java 21+)
# JdbcTemplate, @Async, RestClient also benefit automatically
```

Q92: What is Spring Modulith?

Answer: Spring Modulith (Spring Boot 3.x) is a new project for building modular monolith applications. It provides: 1) Structured modules with explicit dependencies (using packages as modules). 2) Automatic architectural verification (tests that modules don't have circular

dependencies). 3) Module-aware integration testing. 4) Published domain events between modules. 5) Actuator endpoint showing module structure. A modular monolith is easier to split into microservices later.

Q93: What is Problem Details (RFC 7807) support in Spring Boot 3?

Answer: Spring Boot 3 adds native support for RFC 7807 Problem Details — a standard JSON error response format. Enable with spring.mvc.problemdetails.enabled=true. Spring MVC automatically returns Problem Details responses for common exceptions (MethodArgumentNotValidException, MethodNotAllowedException, etc.). The response includes: type (URI), title, status (HTTP code), detail, instance fields. This standardizes error responses across services.

```
# Enable Problem Details responses
spring.mvc.problemdetails.enabled=true

# Automatic error response format:
# {
#   "type": "about:blank",
#   "title": "Bad Request",
#   "status": 400,
#   "detail": "name must not be blank",
#   "instance": "/api/users"
# }
```

Section 11: Spring Boot 4.x Features & Changes (Q94–Q100)

Spring Boot 4.0 was released November 20, 2025. It is built on Spring Framework 7.
Minimum Java: 17. Recommended: Java 25 (latest LTS). Jakarta EE 11 (Servlet 6.1, JPA 3.2).
Jackson 3 replaces Jackson 2. GraalVM 25 required for native images.

Q94: What are the key new features and baseline changes in Spring Boot 4.0?

Answer: Spring Boot 4.0 (Nov 2025) key changes: 1) Built on Spring Framework 7 and Jakarta EE 11. 2) Java 17 minimum, Java 25 first-class support. 3) Jackson 3 replaces Jackson 2 as default JSON library (breaking change in date serialization behavior). 4) Complete modularization — Boot codebase split into 70+ focused JARs. 5) JSpecify null-safety annotations adopted portfolio-wide. 6) API Versioning built into Spring MVC and WebFlux. 7) HTTP Service Client auto-configuration. 8) Micrometer 2 + OpenTelemetry Starter. 9) GraalVM 25 required for native images.

Q95: What is API Versioning in Spring Boot 4 / Spring Framework 7?

Answer: Spring Framework 7 introduces first-class API versioning in request mappings. Add version attribute to @GetMapping, @PostMapping etc. Use WebMvcConfigurer.configureApiVersioning() to choose the versioning strategy: URL path segment, request header, or media type parameter. A single controller can handle multiple API versions without code duplication. Supports deprecation headers (Sunset, Deprecation) for versioned endpoints.

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}", version = "1.0")
    public UserDTOv1 getUserV1(@PathVariable Long id) { ... }

    @GetMapping("/{id}", version = "2.0")
    public UserDTOv2 getUserV2(@PathVariable Long id) { ... }
}

@Configuration
public class ApiConfig implements WebMvcConfigurer {
    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        // Strategy: Accept header param: Accept: application/json;version=2.0
        configurer.useMediaTypeParameterVersioning();
        // Or path: /api/v1/users, /api/v2/users
        // configurer.usePathSegment(1);
    }
}
```

Q96: What changed with Jackson in Spring Boot 4 (Jackson 3)?

Answer: Spring Boot 4 upgrades to Jackson 3 as the default JSON library. Key breaking changes: 1) Date serialization default flipped: Jackson 2 wrote dates as Unix timestamps (1699257000000). Jackson 3 writes ISO-8601 strings ("2025-11-06T05:30:00") by default. 2) JSON property ordering default flipped: Jackson 2 preserved insertion order. Jackson 3 sorts alphabetically by default. 3) Jackson 3 requires Java 17+. Migration: audit all LocalDate/LocalDateTime/Instant fields in API responses; update integration tests that assert on JSON format.

Q97: What is JSpecify null-safety and how does Spring Boot 4 adopt it?

Answer: JSpecify is a Google-led cross-vendor standard for Java nullability annotations (JSpecify 1.0, July 2024, supported by JetBrains, Oracle, Meta, Sonar). Spring Boot 4 adopts it portfolio-wide, deprecating Spring's own org.springframework.lang.@NonNull/@Nullable. Key concept: annotate package-info.java with @NullMarked — everything in that package is non-null by default. Only use

@Nullable for fields that can actually be null. IDE tooling (IntelliJ 2025.3+) provides first-class JSpecify support.

```
// package-info.java
@NullMarked
package com.myapp.service;

import org.jspecify.annotations.NullMarked;

// In the service — everything non-null by default
public class UserService {
    public UserDTO findUser(String email) { // non-null by default
        // ...
    }

    public @Nullable UserDTO findUserOrNull(String email) { // explicitly nullable
        // ...
    }
}
```

Q98: What is the Spring Boot 4 codebase modularization?

Answer: Spring Boot 4 splits the monolithic spring-boot-autoconfigure JAR into 70+ focused, smaller JARs (e.g., spring-boot-data-jpa, spring-boot-web-servlet, spring-boot-security, spring-boot-mongodb). Benefits: smaller application footprint (only include modules you need), faster startup, better GraalVM AOT analysis (less unused code), clearer dependency graph. The starter POMs still aggregate the right modules — you add starters as before, but the underlying JARs are now modular.

Q99: What are the Spring Boot 3 to Spring Boot 4 migration steps?

Answer: Migration path: 1) Migrate to Spring Boot 3.5 first (fix all deprecation warnings). 2) Ensure Java 17+ (Java 21+ recommended). 3) Upgrade GraalVM to 25+ if using native images. 4) Update to Jakarta EE 11: Servlet container to Tomcat 11/Jetty 12. 5) Handle Jackson 3 breaking changes: audit date/time field serialization, fix tests asserting on timestamp format or property ordering. 6) Update javax.annotation and javax.inject to jakarta equivalents (Spring Boot 4 drops support). 7) Remove Spring's @NonNull/@Nullable in favor of JSpecify if desired. 8) Test thoroughly — Jackson 3 changes are silent runtime issues.

Q100: What is the Resilience support (Retryable/ConcurrencyLimit) in Spring Boot 4 / Spring Framework 7?

Answer: Spring Framework 7 introduces built-in resilience annotations: @Retryable for automatic retry with exponential backoff, and @ConcurrencyLimit for limiting parallel executions. Enable with @EnableResilientMethods. @Retryable(maxAttempts=4, delay=500, multiplier=2.0) automatically retries a failed method with 500ms, 1s, 2s, 4s delays. @ConcurrencyLimit(2) ensures at most 2 concurrent executions of a method at any time. These replace the need for Resilience4j for common resilience patterns.

```
@Configuration
@EnableResilientMethods
public class ResilienceConfig { }

@Service
public class ExternalApiService {

    @Retryable(
        maxAttempts = 4,
        delay = 500,
        multiplier = 2.0,   // 500ms, 1s, 2s, 4s backoff
        maxDelay = 5000,
        jitter = 100        // prevents thundering herd
    )
    public String fetchData(String id) {
        return externalApi.getData(id); // retried on exception
```

```
    }

    @ConcurrencyLimit(2)  // max 2 parallel executions
    public String heavyOperation(String id) { ... }
}
```

Quick Reference: Spring Boot Version Comparison

Feature / Area	Spring Boot 2.x	Spring Boot 3.x	Spring Boot 4.x
Java Baseline	Java 8+	Java 17+	Java 17+ (Java 25 preferred)
Spring Framework	5.x	6.x	7.x
Jakarta EE	javax.* (EE 8)	jakarta.* (EE 9/10)	jakarta.* (EE 11)
JSON Library	Jackson 2	Jackson 2	Jackson 3 (breaking changes!)
Auto-config file	spring.factories	AutoConfiguration.imports	AutoConfiguration.imports
Security Config	WebSecurityConfigurerAdapter	SecurityFilterChain @Bean	SecurityFilterChain @Bean
Native Images	Limited	First-class GraalVM support	GraalVM 25, AOT Repositories
Observability	Spring Cloud Sleuth	Micrometer Tracing built-in	Micrometer 2 + OTEL Starter
Virtual Threads	Not supported	Java 21 + spring.threads.virtual	Auto with virtual threads
HTTP Clients	RestTemplate / WebClient	RestClient / @HttpExchange	HTTP Service Client auto-config
API Versioning	Manual (URL hack)	Manual	First-class in MVC/WebFlux
Null Safety	Spring @NonNull	Spring @NonNull	JSpecify @NullMarked (standard)
Testing	@SpringBootTest, MockMvc	+ Testcontainers @ServiceConnection	+ RestTestClient
Problem Details	Not built-in	spring.mvc.problemdetails.enabled	Enhanced ProblemDetail support
Resilience	Resilience4j (manual)	Resilience4j (manual)	@Retryable @ConcurrencyLimit built-in
JUnit Support	JUnit 4 + 5	JUnit 5 only	JUnit 5 only (JUnit 4 deprecated)
Codebase	Monolithic autoconfigure	Monolithic autoconfigure	Modular (70+ focused JARs)
Recommended JDK	Java 11 or 17	Java 17 or 21	Java 25 (LTS)

⚠ Spring Boot 3.x requires Spring Boot 2.x → 3.x migration: all javax.* imports MUST become jakarta.*
⚠ Spring Boot 4.x requires Jackson 3: date fields serialized differently — audit all API responses!
⚠ Recommended migration path: 2.x → 3.5 (fix deprecations) → 4.0
⚠ Spring Boot 3.5.x OSS support ends June 2026 — plan upgrade to 4.x